

오늘의 작업기록 — 2026년 6월 23일

mini-ledger 증분 2 완료: transfer + test Main

(모든 오답 전면 해부 + 내 코드 vs 정답 풀코드 비교 + 오답노트)

한눈에

항목	내용
목표	증분 2 마무리 — transfer 구현 + Main 으로 실행 증명
결과	둘 다 완료 ✓ — transfer 작동(1100/4500 출력) + 커밋·푸시
커밋	9cc49e6 "implement transfer and test Main" (2 files changed, Main.java 신규)
푸시	9da10ce..9cc49e6 main -> main
저장소	github.com/Taewoo-Won/mini-ledger
오늘 틀린 횟수	16곳 (transfer 3 + Main 시도1 10 + Main 시도2 3) + heredoc 사고 1
핵심 성취	transfer를 답 안 받고 내 손으로 짰다 = 증분 2의 개념적 심장
학습 방식	미리 챕터 읽기 폐기 → 막힐 때 그 한 줄만 (just-in-time)

오늘은 많이 틀렸다. 근데 틀린 걸 전부 복기할 수 있다는 게 오늘의 진짜 수확이다. 베꼈으면 못 가졌을 오답노트다. 아래는 내가 틀린 모든 것의 해부다.

0. 출발점 — 오늘의 규칙

transfer는 이 프로젝트에서 답을 받으면 안 되는 단 하나였다. - getAccount는 받아도 손해가 적다. 하지만 transfer는 증분 2의 개념적 심장이고, mini-ledger가 토이가 아닌 진짜 이유다. 베끼면 가장 값진 한 번을 통째로 건너뛴다. - 좋은 소식: 오늘은 새 개념이 0개. transfer에 필요한 도구를 전부 갖고 있었다 → getAccount ×2, withdraw , deposit . 배울 게 없으니 핑계도 없었다.

결과: 답 안 받고 짰다. 세 번 틀렸지만 전부 스스로 고쳤다.

PART 1 — transfer

1-1. transfer가 하는 일

transfer(fromId, toId, amount) = from 계좌 → to 계좌로 amount 만큼 옮긴다. 안전하게.

안전한 순서 (이게 전부다):

- fromId로 계좌 가져오기 (없으면 throw)
- toId로 계좌 가져오기 (없으면 throw) ← 두 계좌 검증 여기서 끝
- from에서 출금
- to에 입금

쓸 도구 4개 전부 이미 내 손에 있었다.

1-2. transfer 시도 1 — 내가 짰 것 (원문)

```
public void transfer(String fromId, String toId, long amount) {
    Account fromAccount = accounts.get(fromId);
    Account toAccount = accounts.get(toId);
    toAccount.deposit(long amount);
    fromAccount.withdraw(long amount);
}
```

틀린 곳 3개

#	내가 쓴 것	무엇이 틀렸나	왜
M1	<code>deposit(long amount)</code>	호출에 타입 키워드 <code>long</code> 을 붙임	<code>long</code> 은 변수를 <i>처음 선언할 때만</i> 쓴다. <i>이미 있는 변수를 넘길 때</i> 는 타입을 안 붙인다. (<code>accounts.put(id, account)</code> 에서 <code>id</code> 에 <code>String</code> 안 붙였던 것과 같다.) → <code>deposit(amount)</code>
M2	<code>accounts.get(fromId)</code>	검증 없는 직접 조회	<code>accounts.get</code> 은 계좌가 없으면 null 반환 (예외 아님). 그럼 다음 줄에서 <code>null.withdraw(...)</code> → 터진다 + 돈 증발 방지가 안 된다. → 내가 만든 검증 메서드 <code>getAccount(fromId)</code> 를 써야 한다.
M3	<code>deposit</code> → <code>withdraw</code> 순서	입금이 출금보다 먼저	거꾸로다. 입금부터 하면, from 잔액 부족으로 withdraw가 throw될 때 <i>to엔 이미 돈이 들어갔는데 from에선 안 빠진다</i> → 돈이 복사된다. (PART 2에서 깊게.)

1-3. transfer 시도 2 — 정답 (원문)

```
public void transfer(String fromId, String toId, long amount) {
    Account fromAccount = getAccount(fromId);
    Account toAccount = getAccount(toId);
    fromAccount.withdraw(amount);
    toAccount.deposit(amount);
}
```

세 개 전부 내가 고쳤다: - `accounts.get` → `getAccount` (검증됨) ✓ - `long amount` → `amount` (타입 제거) ✓ - 출금 → 입금 순서 ✓

1-4. 플코드 비교 — 시도1 vs 정답

시도 1 (틀림)

`| Account fromAccount = accounts.get(fromId);|`
`| Account toAccount = accounts.get(toId); |`
`| toAccount.deposit(long amount); |`
`| fromAccount.withdraw(long amount); |`

↑ 검증X ↑ 타입붙임 ↑ 순서거꾸로

정답

`| Account fromAccount = getAccount(fromId);|`
`| Account toAccount = getAccount(toId); |`
`| fromAccount.withdraw(amount); |`
`| toAccount.deposit(amount); |`

↑ 검증0 ↑ 변수만 ↑ 출금먼저

PART 2 — ★ 개념의 심장: "왜 출금이 입금보다 먼저인가" ★

오늘 미룬 진짜 시험이다. 코드는 옳게 짰으니, 이제 **말로 꺼낼 수 있어야** 한다. (면접 꼬리질문에서 사는 능력.) 아래가 완전한 추론이다 — 이걸 공부해서 네 말로 만들어라.

2-1. transfer가 막아야 할 두 가지 제약

- 제약 ① — 존재하지 않는 계좌: to 계좌가 없는데 from에서 먼저 돈을 빼면 → from 돈은 사라지고 갈 곳은 없다 = 돈 증발.
 - 제약 ② — 잔액 부족: from 잔액이 부족한데 to에 먼저 입금하면 → to엔 돈이 생기고 from은 안 줄었다 = 돈 복사. (시스템 전체 잔액이 늘어남.)
- 안전한 순서는 이 둘을 *동시에* 막는다.

2-2. 왜 "검증을 먼저" 하면 제약 ①이 막히나

```
Account fromAccount = getAccount(fromId);    // 없으면 여기서 throw
Account toAccount = getAccount(toId);        // 없으면 여기서 throw
// ↑ 두 줄을 '돈 건드리기 전에' 끝낸다
fromAccount.withdraw(amount);                 // 여기 도달했다 = 두 계좌 다 존재 확정
toAccount.deposit(amount);
```

- `getAccount` 는 계좌가 없으면 throw한다 (어제 내가 짰 가드).
- 두 계좌 검증을 **출금·입금 전에** 끝내므로, 둘 중 하나라도 없으면 → throw → withdraw/deposit은 **실행조차 안 됨** → **아무 잔액도 안 변함**.
- 만약 `accounts.get` 을 직접 썼다면? get은 없어도 throw 안 하고 null을 준다 → 검증이 *미뤄지고*, 운 나쁘면 한쪽 잔액이 변한 뒤 `null` 에서 터진다. → 그래서 M2가 위험했다.

2-3. 왜 "출금을 먼저" 하면 제약 ②가 막히나 (← 핵심)

열쇠: 출금은 **실패할 수 있는** 단계, 입금은 **항상 성공하는** 단계다.

- `withdraw` 는 잔액 부족이면 throw한다 — 그리고 *빼기 전에* 검사한다 (어제 가드: `if (balance < amount) throw`). → 실패해도 from 잔액 안 변함.
- `deposit` 은 (amount>0이면) **실패할 일이 없다**. 게다가 withdraw가 먼저 `amount>0` 을 통과시켰으니, 출금이 성공한 뒤의 입금은 **반드시 성공**한다.

시나리오 비교 (잔액 부족일 때):

■ 출금 먼저 (정답):
withdraw → 잔액 부족 → throw → 멈춤
결과: from 안 변함, deposit 실행 안 됨, to 안 변함 → ✓ 아무것도 안 변함

■ 입금 먼저 (M3, 틀림):
deposit → to에 돈 들어감 ✓
withdraw → 잔액 부족 → throw → 멈춤
결과: to는 +amount 됐는데 from은 그대로 → ✗ 돈 복사

→ 원리: "실패할 수 있는 일을, 돌이킬 수 없는 일보다 먼저 하라." 위험한 단계(출금)를 앞세워, 실패하면 *그 뒤가 아예 실행 안 되게* 한다.

2-4. 한 줄 요약 (이걸 외워라)

"출금이 유일하게 실패할 수 있는 단계라서 먼저 둔다. 출금이 막히면 입금은 아예 실행되지 않아, 둘 다 성공하거나 아무것도 안 변하거나 — 둘 중 하나만 일어난다."

이게 나중에 DB에서 배울 트랜잭션·원자성(atomicity)의 손맛 버전이다. 지금 코드로 이미 그 감각을 깔았다.

PART 3 — Main (실행 증명 장치)

transfer가 *진짜* 도는지 보려면 굴러야 한다. Main도 내가 짰다. 여기서 제일 많이 틀렸다 (시도1에서 10곳). 전부 해부한다.

3-1. Main이 하는 일

- Bank 하나 만들기
- 계좌 2개 생성
- transfer 호출
- 두 잔액 출력 → 숫자가 맞나 **눈으로 확인**

3-2. Main 시도 1 — 내가 짰 것 (원문 그대로)

```
public class Main {  
    public static void main(String[] args) {  
        Bank bank = new Bank(id, owner, long initialBalance);  
        createAccount("a", owner, initialBalance);  
        createAccount("b", owner, initialBalance);  
    }  
  
    transferAccount fromAccount = getAccount("a", "1000원");  
    transferAccount toAccount = getAccount("b", "1000원");  
    bank.getAccount("a") .getBalance();  
    bank.getAccount("b") .getBalance();  
    System.out.prinlin()  
}  
}
```

틀린 곳 — 무려 10개

#	내가 쓴 것	무엇이 틀렸나	정답 방향
M4	(transfer 호출 없음)	Main의 목적인 transfer가 빠짐	bank.transfer("a", "b", 1000); 추가
M5	createAccount(...)	bank. 없이 떠 있음 — 누가 시키나?	createAccount 는 Bank의 메서드 → bank.createAccount(...) (객체.메서드)
M6	new Bank(id, owner, long initialBalance)	생성자에 인자 + 타입까지 넣음	Bank는 빈 그릇 → new Bank() (괄호 비움). id·주인·잔액은 계좌정보지 Bank 정보가 아님
M7	transferAccount	존재하지 않는 타입	그냥 삭제. 계좌 타입은 Account
M8	getAccount("a", "1000원")	getAccount에 인자 2개 (1개만 받음)	getAccount(String id) — id 하나만. 금액 안 받음
M9	"1000원"	금액을 문자열로 표현	금액은 숫자 → 1000 (따옴표·"원" 제거)
M10	prinlin	철자 틀림 (i 하나 빠짐)	println (p-r-i-n-t-l-n)
M11	prinlin()	빈 괄호 — 출력할 내용 없음	안에 잔액을 넣어야 함
M12	System.out.prinlin()	세미콜론 ; 없음	문장 끝 ;
M13	중간의 }	main 종괄호가 중간에 닫혀 나머지가 밖으로 튀어나감	모든 코드가 main { } 안에

시도 1의 진짜 병: "누가 누구에게 시키는가"를 놓쳤다. createAccount·getAccount·transfer는 전부 **bank**가 가진 도구 → 전부 **bank**. 를 붙여야 한다. 이 하나만 잡으면 절반이 풀린다.

3-3. Main 시도 2 — 내가 짠 것 (원문)

bank. 붙이고, **new Bank()** 비우고, transfer 넣었다. 4곳 남았다.

```
public class Main {
    public static void main(String[] args) {
        Bank bank = new Bank();
        bank.createAccount("a", "원태우", 2100);
        bank.createAccount("b", "원태순", 3500);
        bank.transfer("a" ,"b" 1000);
        System.out.println(bank.getAccount("a").getBalance(1100));
        System.out.println(bank.getAccount("b").getBalance(4500));
    }
}
```

틀린 곳 4개

#	내가 쓴 것	무엇이 틀렸나	정답
M14	transfer("a" ,"b" 1000)	"b" 와 1000 사이 심표 누락	인자는 전부 심표로 → transfer("a", "b", 1000)
M15	println	대문자 I (아이) — 폰이 l/i 헷갈림	소문자 l (엘) → println
M16	getBalance(1100)	getBalance에 숫자를 넣음 (개념 오해)	getBalance() — 괄호 비움. 1100을 넣는 게 아니라, getBalance가 돌려주는 값이 1100인지 확인하는 것

M16이 오늘 가장 중요한 개념 오류다. **getBalance()** 는 잔액을 물어보는 것 — *"얼마야?"*. 내가 1100을 써넣으면 그냥 1100을 출력할 뿐, transfer가 맞는지 *증명이 안 된다*. 빈 **getBalance()** 가 진짜 잔액을 꺼내와야 그게 2100-1000=1100인지 *프로그램*이 보여준다. 코드에 정답을 적는 게 아니라, 프로그램이 정답을 계산하게 두는 것.

3-4. Main 시도 3 — 정답 (원문)

```
public class Main {
    public static void main(String[] args) {
        Bank bank = new Bank();
        bank.createAccount("a", "원태우", 2100);
        bank.createAccount("b", "원태순", 3500);
        bank.transfer("a", "b", 1000);
        System.out.println(bank.getAccount("a").getBalance());
        System.out.println(bank.getAccount("b").getBalance());
    }
}
```

3-5. 풀코드 비교 — Main 시도1(틀림) vs 정답

— 시도 1 (10곳 틀림) —————

```
public class Main {
    public static void main(String[] args){
        Bank bank = new Bank(id,owner,long ...);
        createAccount("a", owner, ...);
        createAccount("b", owner, ...);
    }
    ← 중괄호 오류

    transferAccount fromAccount =
        getAccount("a","1000원");
    transferAccount toAccount =
        getAccount("b","1000원");
    bank.getAccount("a").getBalance();
    bank.getAccount("b").getBalance();
    System.out.printrin()    ← 철자·빈괄호·;

}
}
```

— 정답 —————

```
public class Main {
    public static void main(String[] args){
        Bank bank = new Bank();
        bank.createAccount("a","원태우",2100);
        bank.createAccount("b","원태순",3500);
        bank.transfer("a","b",1000);
        System.out.println(
            bank.getAccount("a").getBalance());
        System.out.println(
            bank.getAccount("b").getBalance());
    }
}
```

핵심 차이 요약: ① **bank**. 전부 붙임 ② **new Bank()** 빈 괄호 ③ transfer 한 줄로 핵심 동작 ④ **getBalance()** 빈 괄호로 *진짜*잔액 조회 ⑤ **println** 정확한 철자 ⑥ 존재하지 않는 **transferAccount** 제거.

PART 4 — 디버깅 케이스: 깨진 heredoc & 무해한 bash 에러

코드는 다 맞았는데 터미널에서 한바탕 사고가 났다. 이것도 복기 가치가 크다.

- **증상:** Bank.java를 `cat > Bank.java << 'EOF'` 로 붙이는데, 화면에 중간이 깨져 나옴: `return account; javac *.javacount.deposit(amount);t);t(toId);Id);Id,` 이어서 무서운 에러들: `-bash: return: account: numeric argument required -bash: return: can only `return' from a function or sourced script -bash: syntax error near unexpected token `('`
- **진단:** 폰에서 여러 줄 heredoc을 붙이다 입력이 **뒤섞였다** — 뒤줄 글자들(`javac *.java` , `deposit(amount)` , `getAccount(toId)` 등)이 한 줄로 엉켜 EOF가 깨끗이 안 닫힘. 닫히지 않은 찌꺼기를 셸이 *명령어로* 해석 → bash 에러.
- **근본 원인:** 모바일 키보드의 멀티라인 붙여넣기 깨짐. (코드 잘못이 아니라 *입력 채널* 문제.)
- **왜 그래도 무사했나 (중요):** 바로 직전 `java Main` 에서 **1100/4500이 떴다**. Main은 `bank.transfer` 를 부르고 transfer는 Bank 안에 있다 → **Bank가 깨졌으면 1100/4500이 절대 안 나온다**. 즉 *이전에 정상 컴파일된 Bank.class가 살아있었고*, 깨진 cat은 EOF 미달힘으로 그냥 무시됐다.
- **확인:** `cat Bank.java` → transfer까지 멀쩡한 정상 파일 확인 (찌꺼기 없음). 안전.
- **교훈 3개:** 1. 수상한 붙여넣기 뒤엔 **cat 파일명** 으로 내용을 직접 확인하라. 화면 잔해 말고 *파일 실제 상태*를 봐야 한다. 2. **java** 가 정상 출력하면, 컴파일된 클래스는 멀쩡하다는 증거다. 3. heredoc 찌꺼기에서 나오는 bash 에러(`return: ...` , `syntax error near (`)는 *셸이 코드 조각을 명령어로 오해한 것* — 무해하다. 당황하지 말 것.

5. 산출물 (깃허브 반영까지)

- **파일 (~/mini-ledger/):**
- `Bank.java` — `transfer` 추가 (createAccount → getAccount → transfer)
- `Main.java` — **신규 생성** (실행 증명 장치)
- `Account.java` — 변경 없음 (어제까지)
- **컴파일·실행:** `bash cd ~/mini-ledger javac *.java && java Main # → 1100 # → 4500`
- **커밋·푸시:** `bash git add . && git commit -m "implement transfer and test Main" && git push [main 9cc49e6] implement transfer and test Main 2 files changed, 17 insertions(+) create mode 100644 Main.java ... 9da10ce..9cc49e6 main -> main`
- **커밋 계보:** `9e671d6` (Account) → `e02b204` (Bank createAccount) → `9cc49e6` (transfer + Main) ✓

6. 실행 증명 — 1100 / 4500 이 뜻하는 것

```
a: 2100 - 1000 = 1100   ✓
b: 3500 + 1000 = 4500   ✓
```

이 두 숫자가 **출력으로** 떴다 = transfer가 진짜로 돈을 옮겼다는 증거다. **내가 코드에 써넣은 게 아니라, 프로그램이 계산해서 보여준 것** (M16에서 배운 핵심). 이게 증분 2의 메인 증명(①).

(아직 안 본 것: ② 없는 계좌 이체 ③ 잔액 부족 이체 → 예외가 throw되는지. 이걸 다음 증분 3에서 try/catch와 함께.)

7. 다음 (증분 3)

1. `IllegalArgumentException` / `IllegalStateException` → **커스텀 예외**로 교체: - `AccountNotFoundException` (계좌 없음) - `InsufficientBalanceException` (잔액 부족)
2. Main에 **try/catch** 도입 → ②③를 *프로그램 안 죽이고* 확인: - ② 없는 계좌 이체 → 예외 잡고 **"from 잔액이 그대로인가"** 깔끔히 검증 (← 돈 증발 방지 최종 증명) - ③ 잔액 부족 이체 → 예외 잡기
3. 여기서 끝는다. (그 이후: 증분 4 enum + Transaction 기록.)

8. 개념 사전

- **메서드 호출 시 타입 안 붙임:** 변수를 *선언*할 때만 `long` / `String` 등을 쓴다. *이미 있는 변수를 넘길* 때는 이름만. (M1) — `deposit(amount)` , `put(id, account)` .
- **객체.메서드 (누가 시키나):** 메서드는 *주인 객체*에게 시킨다. `bank.createAccount(...)` , `from.withdraw(...)` . 혼자 떠 있으면 안 됨. (M5)
- **생성자 빈 괄호:** `new Bank()` — Bank는 인자 없이 태어나는 빈 그릇. (M6) 만들 때 뭘 넣을지는 *그 클래스의 생성자*가정한다.
- `getBalance()` 는 질문, **괄호 비움:** 값을 *돌려받는* 메서드. 숫자를 넣는 게 아니라 *받아서 확인*. (M16) — 가장 중요한 개념 교정.
- `println` 철자: `p-r-i-n-t-l-n` . 소문자 **l** (엘), 대문자 **I** (아이) 아님. 폰에서 특히 조심. (M10, M15)
- **존재하지 않는 타입 금지:** 계좌는 `Account` . `transferAccount` 같은 거 없음. (M7)
- **문자열 vs 숫자:** 금액은 숫자 `1000` , `"1000원"` (문자열) 아님. (M9)
- **검증 먼저 (확장):** "변경 전에 검사"가 두 계좌로 확장된 게 transfer. (PART 2)
- **위임:** transfer는 `amount<=0` 을 재검사 안 함 — withdraw/deposit이 이미 함. (어제 `this.id` 깨달음과 같은 결.)
- **heredoc (cat > 파일 << 'EOF' ... EOF):** 터미널에서 파일 통째로 저장. 폰에선 멀티라인 붙여넣기가 깨질 수 있으니 직후 `cat 파일명` 으로 확인. (PART 4)

9. 비유 모음

- **transfer 순서 = 외나무다리 건너기:** 떨어질 수 있는 쪽(출금=잔액부족 위험)을 *먼저* 건넌다. 거기서 떨어지면(throw) 아직 집(입금)을 안 옮겼으니 손해가 없다. 안전하게 건너 뒤에야 짐을 옮긴다(입금).

- **bank.** = 명령의 주어: "(은행아) 계좌 만들어"처럼 *누구에게* 시키는지 빠지면 허공에 대고 말하는 것. 자바는 허공을 못 알아듣는다.
- **getBalance()** 빈 괄호 = 저울: 저울에 "1100"이라고 *써붙이*는 게 아니라, 물건을 올려 저울이 *가리키*는 숫자를 읽는 것. 내가 답을 적으면 증명이 아니다.
- **깨진 heredoc** = 편지 봉투 안 닫음: 봉투(EOF)를 안 닫으면 편지지(코드)가 쏟아져 바닥(셀)에 흩어지고, 누군가 그 조각을 *명령*으로 오해한다. 그래도 원본 파일은 책상에 멀쩡히 있다 — **cat** 으로 확인하면 된다.

10. ★ 오답노트 — 오늘 틀린 16곳 총정리 (이걸 공부해라) ★

#	어디서	틀린 코드	고친 코드	한 줄 규칙
M1	transfer	deposit(long amount)	deposit(amount)	넘길 땐 타입 안 붙인다
M2	transfer	accounts.get(fromId)	getAccount(fromId)	조회는 검증 메서드로 (null 방지)
M3	transfer	deposit 먼저	withdraw 먼저	실패할 수 있는 일을 먼저
M4	Main①	(transfer 없음)	bank.transfer("a","b",1000)	Main의 목적을 빠뜨리지 말 것
M5	Main①	createAccount(...)	bank.createAccount(...)	객체.메서드 — 주어를 붙여라
M6	Main①	new Bank(id,owner,...)	new Bank()	생성자가 안 받으면 괄호 비움
M7	Main①	transferAccount	Account	없는 타입 쓰지 말 것
M8	Main①	getAccount("a","1000원")	getAccount("a")	인자 개수는 정의대로 (1개)
M9	Main①	"1000원"	1000	금액은 숫자, 문자열 아님
M10	Main①	println	println	철자 p-r-i-n-t-l-n
M11	Main①	println() 빈 괄호	println(...값...)	출력엔 내용을 넣어라
M12	Main①	; 없음	; 붙임	문장 끝 세미콜론
M13	Main①	} 중간에 닫힘	main { } 안에 전부	중괄호 짝·범위 확인
M14	Main②	"b" 1000	"b", 1000	인자는 전부 침표로 구분
M15	Main②	printIn (대문자 I)	println (소문자 l)	l(엘) vs I(아이) 구분
M16	Main②	getBalance(1100)	getBalance()	질문 메서드는 괄호 비움 — 답을 적지 마라
(사고)	터미널	깨진 heredoc	cat 파일명 으로 확인	수상한 붙여넣기 뒤엔 파일 검증

테마별 묶음 (패턴으로 기억): - "누가 시키나" (객체.메서드): M5 — 가장 자주 놓침. - 메서드 호출 문법 (타입·침표·인자 수): M1, M8, M14. - 생성자/타입/값: M6, M7, M9. - 메서드의 *의미*: M16 (getBalance는 질문), M2 (getAccount는 검증). - 순서·안전: M3 (출금 먼저). - 철자·구두점: M10, M11, M12, M13, M15.

11. 회고

오늘 16곳 틀렸다. 근데 **transfer를 답 안 받고 짰다** — 이 프로젝트에서 베끼면 안 되는 단 하나를 지켰다. 그게 오늘의 전부다.

틀린 패턴이 보인다: 대부분 *문법*(타입 안 붙이기, **bank.** 붙이기, 침표, 철자)이고, 진짜 *개념* 오류는 둘이었다 — M2(조회는 검증 메서드로)와 M16(질문 메서드는 괄호 비움). 문법은 손에 붙으면 사라진다. 개념 둘은 이제 안다.

학습 방식도 오늘 바꿨다: **미리 챕터 읽기는 폐기**. 막힐 때 그 한 줄만 찾는다. "15페이지 읽었는데 안 연결돼"가 안 생기게.

다음엔 증분 3 — 커스텀 예외 + try/catch. 그때 ②③(돈 증발/복사 방지)를 *프로그램 안 죽이고* 최종 증명한다. 도구는 또 거의 다 갖고 시작한다.

한 줄: 많이 틀렸다는 건 많이 짰다는 거다. 그리고 틀린 걸 전부 적어놨으니, 같은 데서 두 번은 안 막힌다.